

Inigo Quilez :: articles :: minimal code for splines - 2003



This is an coding trick I developed for the **Paradise** 64k demo in 2004. Before Paradise, all the camera movents of our 4k and 64k intros had been done with simple combinations of sinus and cosinus functions. However we wanted more camera control for Paradise, so we decided to implement some minimal spline functionality. We decided that Catmull-Rom splines, even if they don't provide tangent controls, would suffice for our little production. So we went for it and we ended up with a cute tiny implementation that allowed us to interpolate both 3D and n-dimensional control points.

Catmull-Rom splines

The beauty of Catmull-Rom splines over Bezier or other types of Splines and other cubic polynomial interpolation functions in general is that with this type of spline control over the shape of the curve is super simple and can't be more intuitive. Simply choose a few point in space, a time for every one of them, *et voilà*, you are done, your path will pass thru those points at those moments in time. Can't be more easy.

I'm not going to explain how Catmull-Rom splines are constructed, you can simply go to google for that, or derive the formulas yourself, it's very easy because you simply have to:

1. build a generic cubic polynomial $p(t) = a + b \cdot t + c \cdot t^2 + d \cdot t^3$, where $p(t)$ is your path in space and a, b, c, d are coefficients (points in space too)
2. take it's derivative $p'(t) = b + 2c \cdot t + 3d \cdot t^2$
3. make sure that at $t=0$ the curve passes thru your first point p_1 (you got the first coefficient already, $a = p_1$)
4. make sure that at $t=0$ the curve has tangent $(p_2 - p_0)/2$ (and you got the second coefficient, $b = (p_2 - p_0)/2$)
5. make sure that at $t=1$ the curve passes thru the second point p_2
6. make sure that at $t=1$ the curve has tangent $(p_3 - p_1)/2$
7. solve 5 and 6 to get c and d

If you follow these steps you will arrive to something like:

$$a = (2 \cdot p_1)$$

```

b = (p2-p0)
c = (2·p0 - 5·p1 + 4·p2 - p3)
d = (-p0 + 3·p1 - 3·p2 + p3)

```

or if you want as

```

a = < { 0, 2, 0, 0 }, {p0,p1,p2,p3} >
b = < {-1, 0, 1, 0 }, {p0,p1,p2,p3} >
c = < { 2,-5, 4,-1 }, {p0,p1,p2,p3} >
d = < {-1, 3,-3, 1 }, {p0,p1,p2,p3} >

```

Mathematicians will tell me you cannot dot a vector with a vector of vectors, but well, you get the idea.

Of course one first needs to know the segment (p1,p2) in which we want to perform the cubic interpolation above. Then, the formula for p(t) have to be used after normalizing t to the proper (t1,t2) interval.

The code

This code is written using standard C types only, so it should be pretty much CopyPaste-able, and easily adapted to your needs. There might be some corner cases you might want to add checks for, like the cases where t is less than zero or bigger than the time of the last key of the spline.

```

static signed char coefs[16] = {
    -1, 2,-1, 0,
    3,-5, 0, 2,
    -3, 4, 1, 0,
    1,-1, 0, 0 };

void spline( const float *key, int num, int dim, float t, float *v )
{
    const int size = dim + 1;
    // find key
    int k = 0; while( key[k*size]<t ) k++;
    // interpolant
    const float h = (t-key[(k-1)*size])/(key[k*size]-key[(k-1)*size]);
    // init result
    for( int i=0; i<dim; i++ ) v[i] = 0.0f;
    // add basis functions
    for( int i=0; i<4; i++ )
    {

```

```
int kn = k+i-2; if( kn<0 ) kn=0; else if( kn>(num-1) ) kn=num-1;
const signed char *co = coefs + 4*i;
const float b = 0.5f*((co[0]*h + co[1])*h + co[2])*h + co[3]);
for( int j=0; j<dim; j++ ) v[j] += b*key[kn*size+j+1];
}
}
```

This might compile to something between 100 or 150 bytes depending on your compiler settings. It can probably be optimized by computing `k*size` only once and reusing it in the rest of the code, and using offsets to index in the `kn` keys, but I don't want to make the code that unreadable here, although I would certainly do it if I was using this for a 4k intro.

As you can guess from the code, the format for the path key is: { t0, x1, y1, z1, t1, x1, y1, z1, t2, ,x2, y2, z2, ... }, that is not the best option regarding data compression, but works allows a small implementation of the spline code. You probably want to store the splines in a more compressible format (like independant streams for the different t, x, y, z signals, and then apply some linear prediction on them and then quantify them for example, or any other thing you might come with) and then convert it to this format before performing the spline evaluation.

inigo quilez - learning computer graphics since 1994